

CONTENTS

Álgebra de Boole y Circuitos Combinatorios	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este problema	
1. Axiomas del álgebra de Boole y teoremas derivados	
2. Funciones booleanas: tabla de verdad, Σ y Π	
3. Simplificación algebraica vs. Mapa de Karnaugh	
3.1 Simplificación algebraica	
3.2 Método de Karnaugh — procedimiento paso a paso	
3.3 El mínimo en dos niveles NO es único	
4. Compuertas lógicas básicas y conjuntos completos	
4.1 Construir AND y OR usando solo NAND	
5. Bloques constructivos estándar	
5.1 Decodificador	
5.2 Multiplexor (MUX)	
5.3 Demultiplexor	
5.4 Multiplexor como generador de funciones (MUX como look-up table)	
6. Sumador completo de 1 bit y sumador de n bits	
6.1 Half adder (semisumador)	
6.2 Full adder (sumador completo)	

- 6.3 Sumador de n bits en cascada (ripple carry)
- 7. Caso resuelto — MUX como look-up table (verificado completo)
- 8. Ejercicios de repaso rápido (con respuesta)
- 9. Resumen ultra-rápido (2 minutos)

Álgebra de Boole y Circuitos Combinatorios

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelaR · Módulo 2

Axiomas y teoremas · Σ/Π y tabla de verdad · Simplificación algebraica y Karnaugh ·
Compuertas y conjuntos completos · Decodificador, multiplexor, demultiplexor · Sumador completo

Cómo usar esta guía: cada sección tiene una definición corta, el procedimiento paso a paso, al menos dos ejemplos resueltos con verificación numérica explícita, y los detalles que suelen explicarse rápido en clase pero que definen si un ejercicio de examen sale bien o mal. Amarillo = ejemplos resueltos y verificados. Verde = trucos de verificación cruzada. Rojo = errores frecuentes de tolerancia cero. Gris = notas generales. Todos los ejemplos numéricos de esta guía fueron verificados de forma independiente por tabla de verdad completa (fuerza bruta) o por un algoritmo propio de Quine-McCluskey, no solo “a ojo”.

0. Por qué existe este problema

Un circuito digital solo tiene dos niveles de tensión: alto y bajo, 0 y 1. Cualquier decisión que un circuito deba tomar — sumar dos números, decodificar una dirección de memoria, elegir qué señal pasar — tiene que expresarse como una función que combina bits de entrada y produce bits de salida. El álgebra de Boole es la herramienta matemática que permite escribir esas funciones, manipularlas algebraicamente, y **traducirlas directamente a compuertas físicas**.

El problema central de este tema tiene tres capas, y cada sección de esta guía ataca una:

1. **¿Cómo describo sin ambigüedad una función de n variables?** → tabla de verdad, Σ , Π (sección 2).
2. **¿Cómo la simplifico para usar menos compuertas/términos?** → axiomas, teoremas, Karnaugh (secciones 1 y 3).
3. **¿Con qué piezas de hardware la construyo?** → compuertas, conjuntos completos, decodificador, multiplexor, sumador (secciones 4 a 7).

Idea que hay que tener siempre presente: una expresión booleana no es “la” función — es una de las muchas formas de escribirla. Dos expresiones con aspecto totalmente distinto pueden ser exactamente la misma función (mismo valor en las 2^n filas de la tabla de ver-

dad). Verificar “¿dan lo mismo en todas las filas?” es la prueba de fuego que nunca falla, aunque sea la más lenta.

1. Axiomas del álgebra de Boole y teoremas derivados

Los 6 axiomas (notación del curso — se citan por número en las demostraciones):

#	Nombre	Enunciado
ax.1	Cierre	$a+b \in B, a \cdot b \in B$
ax.2	Neutro	$a+0=a, a \cdot 1=a$
ax.3	Conmutativa	$a+b=b+a, a \cdot b=b \cdot a$
ax.4	Asociativa	$(a+b)+c=a+(b+c), (a \cdot b) \cdot c=a \cdot (b \cdot c)$
ax.5	Distributiva	$a \cdot (b+c)=a \cdot b+a \cdot c, a+(b \cdot c)=(a+b) \cdot (a+c)$
ax.6	Complemento	$a+\bar{a}=1, a \cdot \bar{a}=0$

Teoremas derivados (se demuestran a partir de los axiomas, no se asumen):

Teorema	Enunciado
Idempotencia	$a+a=a, a \cdot a=a$
Dominación	$a+1=1, a \cdot 0=0$
Absorción	$a+a \cdot b=a, a \cdot (a+b)=a$
Involución (comp. de comp.)	$(\bar{\bar{a}})=a$
De Morgan	$(a+b)\bar{ }=\bar{a} \cdot \bar{b}, (a \cdot b)\bar{ }=\bar{a}+\bar{b}$
Consenso / f.c. (factor común)	$a+\bar{a} \cdot b=a+b$

Error de tolerancia cero #1: en una demostración algebraica, **cada paso debe citar el axioma o teorema exacto que lo justifica**, no “por lógica” o “se simplifica”. El corrector anula el ejercicio si un paso intermedio no tiene justificación válida, aunque el resultado final sea correcto. Escribí siempre el nombre entre paréntesis al lado de cada línea, como en los ejemplos de abajo.

Demostración 1 — $a+1=1$ (dominación, a partir de los axiomas):

$$\begin{aligned}
 a+1 &= a+(a+\bar{a}) \text{ (ax.6, complemento)} \\
 &= (a+a)+\bar{a} \text{ (ax.4, asociativa)} \\
 &= a+\bar{a} \text{ (idempotencia)} \\
 &= 1 \text{ (ax.6) } \blacksquare
 \end{aligned}$$

Verificación por tabla de verdad (2 filas, $a=0$ y $a=1$): $a=0 \rightarrow 0+1=1 \checkmark$; $a=1 \rightarrow 1+1=1 \checkmark$. Se cumple en ambas filas.

Demostración 2 — $a+\bar{a} \cdot b=a+b$ (teorema del consenso):

$$\begin{aligned}
 a+\bar{a}b &= a(b+\bar{b})+\bar{a}b \text{ (ax.2, neutro: } a=a \cdot 1=a(b+\bar{b})) \\
 &= ab+a\bar{b}+\bar{a}b \text{ (ax.5, distributiva)} \\
 &= b(a+\bar{a})+a\bar{b} \text{ (f.c. de } b) \\
 &= b+a\bar{b} \text{ (ax.6)} \\
 &= (b+a) \cdot (b+\bar{b}) \text{ (ax.5)} \\
 &= (b+a) \cdot 1 \text{ (ax.6)} \\
 &= b+a = a+b \text{ (ax.2, ax.3) } \blacksquare
 \end{aligned}$$

Verificación por tabla de verdad (4 filas de a,b): $(0,0) \rightarrow 0+0=0$, $a+b=0 \checkmark$.

$(0,1) \rightarrow 0+1 \cdot 1=1$, $a+b=1 \checkmark$ · $(1,0) \rightarrow 1+0 \cdot 0=1$, $a+b=1 \checkmark$ · $(1,1) \rightarrow 1+0 \cdot 1=1$, $a+b=1 \checkmark$. Coincide en las 4 filas.

Truco de verificación cruzada para cualquier demostración algebraica: si la igualdad tiene n variables, armá la tabla de verdad de las 2^n filas para ambos lados (izquierdo y derecho de la igualdad) y compará fila por fila. Es mecánico, no requiere “creatividad”, y es la forma más confiable de confirmar que una demostración (o su resultado) es correcta antes de entregarla.

Error frecuente #2: usar un teorema derivado (idempotencia, absorción, De Morgan) como si fuera un axioma en un ejercicio que pide “demostrar usando axiomas” **desde cero** — hay que fijarse qué pide el enunciado. Si el ejercicio dice “demostrar que $a+ab=a$ ” y ya se demostró en un ítem anterior, se puede citar como “(demostrado en el ítem anterior)”; pero no se puede asumir sin demostrar un teorema que todavía no se probó en esa misma tanda de ejercicios.

2. Funciones booleanas: tabla de verdad, Σ y Π

Definición: una función booleana de n variables tiene una tabla de verdad de 2^n filas — una por cada combinación posible de valores 0/1 de las n variables, numeradas de 0 a 2^n-1 en orden binario estándar (la variable más a la izquierda es el bit más significativo).

Σ (suma de productos canónicos / minterms): para cada fila donde $f=1$, se arma un producto (minterm) con todas las variables — la variable sin complementar si vale 1 en esa fila, complementada si vale 0 — y se suman (OR) todos esos productos.

Π (producto de sumas canónicas / maxterms): análogo pero sobre las filas donde $f=0$: cada suma (maxterm) lleva las variables sin complementar si valen 0 en esa fila, complementadas si valen 1, y se multiplican (AND) todas esas sumas.

Relación clave entre Σ y Π : son la misma tabla de verdad vista desde dos lados. Los índices de Σ y los índices de Π de una misma función son ****complementarios**** (cubren, entre los dos, todas las filas de 0 a 2^n-1 sin superponerse) — y dentro de cada maxterm de Π , las variables están complementadas ****exactamente al revés**** que en el minterm análogo de Σ .

Ejemplo 1 — $f(x,y,z)=\sigma(1,4,5,6)$: filas 1,4,5,6 en binario (xyz) son 001, 100, 101, 110.

$$f(x,y,z) = \bar{x}\bar{y}z + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z}$$

Verificación (tabla de verdad completa, 8 filas): se evaluó la expresión en las 8 combinaciones de x,y,z y coincide exactamente con $f=1$ solo en las filas 1,4,5,6 — sin minterms de más ni de menos. ✓

Ejemplo 2 — $h(a,b,c)=\pi(2,3,6,7)$: las filas con $h=0$ son 2(010),3(011),6(110),7(111) — todas tienen $b=1$. Las filas restantes (0,1,4,5), con $h=1$, son 000,001,100,101.

$$h(a,b,c) = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + a\bar{b}\bar{c} + a\bar{b}c$$

Verificación (tabla de verdad completa, 8 filas): la expresión vale 1 exactamente en las filas 0,1,4,5 y 0 en 2,3,6,7 — coincide con $\pi(2,3,6,7)$. ✓ Notar además que estas 4 filas en 1 tienen siempre $\bar{b}$, consistente con la observación directa ($b=1$ en todas las filas de π).

Concepto	Toma las filas donde...	Cada término lleva la variable...
Σ (minterms)	$f=1$	sin complementar si vale 1, complementada si vale 0
Π (maxterms)	$f=0$	sin complementar si vale 0, complementada si vale 1

Error frecuente: escribir Σ como producto de sumas o Π como suma de productos por apurarse. Σ siempre es una ****suma de productos**** (OR de ANDs), Π siempre es un ****producto de sumas**** (AND de ORs) — el nombre lo indica al revés de lo que uno esperaría a primera vista, por eso conviene fijar la regla de memoria antes del examen.

3. Simplificación algebraica vs. Mapa de Karnaugh

Objetivo de ambos métodos: llegar a una expresión en **dos niveles** (una capa de ANDs seguida de una capa de OR, o viceversa) con la menor cantidad de términos y de literales posible — no necesariamente la menor cantidad de compuertas físicas (ver recuadro rojo más abajo).

3.1 SIMPLIFICACIÓN ALGEBRAICA

Se aplican los axiomas y teoremas de la sección 1 directamente sobre la expresión, paso a paso, citando cada uno.

Ejemplo 1 — simplificar $a\bar{b}(a+\bar{b})\bar{c}+b$:

$$= a\bar{b}\bar{a}\bar{c}+a\bar{b}\bar{b}\bar{c}+b \text{ (ax.5, distributiva)}$$

$$= 0+a\bar{b}\bar{c}+b \text{ (} a\bar{b}\bar{a}=0 \text{ por ax.6 aplicado a } a\cdot\bar{a}, \text{ ya que } \bar{b}\bar{b}=\bar{b} \text{ por idempotencia)}$$

$$= a\bar{b}\bar{c}+b \text{ (ax.2, neutro)}$$

Esta forma ($a\bar{b}\bar{c}+b$) ya cubre la función pero todavía no es la mínima: agrupando por Karnaugh se ve que el término $a\bar{b}\bar{c}$ está contenido, junto con $ab\bar{c}$ (que no aparece explícito pero queda cubierto por b), dentro de un grupo más grande $a\bar{c}$ — quedando la forma mínima:

Resultado: **$a\bar{c}+b$**

Verificación por tabla de verdad (8 filas de a,b,c): la expresión original $a\bar{b}(a+\bar{b})\bar{c}+b$ y el resultado $a\bar{c}+b$ coinciden exactamente en las 8 filas. ✓

Ejemplo 2 — simplificar $bc+da+c+dc(ab+dc)$:

$$= bc+ad+c+abcd+cd \text{ (ax.5 distribuido, y } dc\cdot dc=dc \text{ por idempotencia)}$$

$$= bc+c+ad+cd(ab+1) \text{ (f.c. de } cd, \text{ agrupando } abcd+cd=cd(ab+1))$$

$$= c+ad+cd \text{ (dominación: } ab+1=1; \text{ y } bc+c=c \text{ por absorción)}$$

$$= c+ad \text{ (absorción: } c+cd=c)$$

Resultado: **$ad+c$**

Verificación por tabla de verdad (16 filas de a,b,c,d): la expresión original y $ad+c$ coinciden exactamente en las 16 filas. ✓

Error frecuente en simplificación algebraica: parar un paso antes de tiempo. Una expresión “parece” simplificada cuando ya no se ve una absorción obvia, pero hay que verificar contra Karnaugh (o tabla de verdad) — si el número de términos/literales de Karnaugh es menor, la simplificación algebraica quedó incompleta y el ejercicio se considera mal resuelto, aunque cada paso individual esté bien justificado.

3.2 MÉTODO DE KARNAUGH — PROCEDIMIENTO PASO A PASO

1. Armar la grilla con las celdas en **código Gray** en filas y columnas (adyacentes difieren en un solo bit) — nunca en orden binario normal.
2. Marcar 1 en cada celda donde $f=1$ (o 0 si se va a agrupar por maxterms).
3. Agrupar los 1's en bloques rectangulares de tamaño **potencia de 2** (1, 2, 4, 8...), lo más grandes posible, permitiendo **adyacencia con wrap-around** (los bordes izquierdo/derecho y superior/inferior de la grilla son adyacentes entre sí).
4. Cada grupo da un término: se leen solo las variables que **no cambian de valor** en ninguna celda del grupo (constantes); las que cambian dentro del grupo se descartan.
5. Sumar (OR) todos los términos de los grupos. Repetir para todos los 1's necesarios (cobertura completa, sin dejar ningún 1 sin agrupar).

Regla práctica para no dejar 1's sueltos: antes de dar la respuesta por terminada, contá que la cantidad de 1's cubiertos por la unión de todos los grupos sea exactamente igual a la cantidad de 1's de la tabla de verdad — ni de más (eso sería agrupar ceros por error) ni de menos.

3.3 EL MÍNIMO EN DOS NIVELES NO ES ÚNICO

Para una misma tabla de verdad puede haber **más de una** expresión mínima válida (mismo número de términos y de literales, pero agrupamientos distintos del mapa). Esto es un resultado conocido del curso y aparece explícitamente en el Práctico 3 (Ejercicio 6).

Ejemplo verificado — $f(a,b,c,d)=\sigma(2,3,7,13,15)$. Mapa de Karnaugh (filas ab, columnas cd, código Gray):

ab\cd	00	01	11	10
00	0	0	1	1
01	0	0	1	0
11	0	1	1	0
10	0	0	0	0

Hay **dos formas distintas** de agrupar los cinco 1's en bloques de tamaño 2, ambas con 3 términos y 9 literales:

Agrupamiento A: $f = bcd + \bar{a}\bar{b}c + abd$

Agrupamiento B: $f = \bar{a}cd + \bar{a}\bar{b}c + abd$

(la diferencia está en cómo se agrupa el par de celdas $ab=00, cd=11$ y $ab=01, cd=11$: junto con $ab=11, cd=11$ forma bcd , o junto con $ab=00, cd=10$ forma $\bar{a}\bar{b}c$ — ambos grupos válidos de tamaño 2, mismo costo)

Verificación (tabla de verdad completa, 16 filas): se evaluaron ambas expresiones en las 16 combinaciones de a, b, c, d — ambas valen 1 exactamente en las filas 2,3,7,13,15 y 0 en el resto, coincidiendo entre sí en las 16 filas. Confirmado computacionalmente (Quine-McCluskey propio) que **no existe** ninguna expresión de 2 niveles con menos de 3 términos/9 literales que cubra esta función: A y B son ambas óptimas y distintas. ■

Error de tolerancia cero: si en un parcial tu agrupamiento de Karnaugh difiere del que dio el docente en la corrección, ****no es automáticamente un error**** — hay que verificar si tu expresión tiene el mismo costo (términos y literales) y si coincide en las 2^n filas de la tabla de verdad. Solo es error si tu expresión (a) no cubre todos los 1's, (b) cubre algún 0, o (c) tiene más términos/literales que el mínimo real (no agrupaste lo más grande posible).

4. Compuertas lógicas básicas y conjuntos completos

Compuertas básicas: AND, OR, NOT, NAND, NOR, XOR, XNOR — conectivas binarias (o unaria, NOT) usadas para implementar circuitos.

Compuerta	Expresión	Sale 1 cuando...
AND	$a \cdot b$	ambas entradas son 1
OR	$a + b$	al menos una entrada es 1
NOT	$\bar{a}$	la entrada es 0
NAND	$(a \cdot b)'$	no ambas son 1 (al menos una es 0)
NOR	$(a + b)'$	ninguna es 1
XOR	$a \oplus b$	las entradas difieren
XNOR	$(a \oplus b)'$	las entradas coinciden

Conjunto de operadores lógicamente completo: un conjunto de compuertas con el cual se puede expresar **cualquier** función booleana combinando únicamente compuertas de ese conjunto. Ejemplos: {AND, OR, NOT}, o el conjunto unitario {NAND}, o {NOR}.

4.1 CONSTRUIR AND Y OR USANDO SOLO NAND

AND con NAND: circuito = NAND(A,B) seguido de un segundo NAND con ambas entradas conectadas a la salida del primero.

Demostración algebraica: sea $Y = (AB)'$. El segundo NAND calcula $(Y \cdot Y)'$.

$(AB)' \cdot (AB)' = (AB) + (AB)'$ (De Morgan, aplicado a $Y \cdot Y = (Y + Y)'$... en realidad se aplica directamente: $NAND(Y, Y) = (Y \cdot Y)' = \bar{Y}$ por idempotencia)

$\bar{Y} = ((AB)')' = AB$ (involución/comp. de comp.)

Resultado: el circuito calcula exactamente **$A \cdot B$** . ■

Verificación por tabla de verdad (4 filas de A,B): NAND(A,B) da 1,1,1,0 para (0,0),(0,1),(1,0),(1,1); el segundo NAND aplicado a ese valor consigo mismo invierte esa salida, dando 0,0,0,1 — exactamente AND. ✓

OR con NAND: circuito = NAND(A,A) y NAND(B,B) (cada uno actúa como inversor al recibir la misma señal en ambas entradas, dando $\bar{A}$ y $\bar{B}$), seguidos de NAND($\bar{A}, \bar{B}$).

Demostración algebraica:

$NAND(A, A) = (A \cdot A)' = \bar{A}$ (idempotencia + complemento)

$NAND(B, B) = (B \cdot B)' = \bar{B}$ (ídem)

$NAND(\bar{A}, \bar{B}) = (\bar{A} \cdot \bar{B})' = \bar{\bar{A} + \bar{B}}$ (De Morgan) = $A + B$ (involución) ■

Verificación por tabla de verdad (4 filas de A,B): el circuito da 0,1,1,1 para (0,0),(0,1),(1,0),(1,1) — exactamente OR. ✓

Truco de verificación cruzada rápido: para probar que {NAND} es completo alcanza con construir NOT, AND y OR con NAND únicamente — una vez que tenés esos tres, cualquier función booleana (que siempre se puede escribir en términos de AND, OR, NOT) queda automáticamente expresable solo con NAND.

Error frecuente: confundir “NAND(A,A) es un inversor” con “cualquier compuerta con las dos entradas iguales es un inversor”. Solo funciona así para NAND y NOR (porque $(A \cdot A) = \bar{A}$ y $(A + A) = \bar{A}$); NAND(A,A) NO es lo mismo que aplicar NOT dos veces ni que anular la señal.

5. Bloques constructivos estándar

5.1 DECODIFICADOR

Definición: circuito con n líneas de entrada y 2^n líneas de salida, donde exactamente **una** salida está en 1 (la correspondiente al valor binario de la entrada) y el resto en 0.

Construcción: la salida Y_i ($i=0 \dots 2^n-1$) es el AND de las n entradas, complementadas o no según el bit correspondiente de i en binario (bit=0 → entrada complementada, bit=1 → entrada sin complementar).

Decodificador 4→16 (entradas w,x,y,z): $Y_0 = \bar{w}\bar{x}\bar{y}\bar{z}$ ($i=0=0000$), $Y_{15} = wxyz$ ($i=15=1111$), $Y_6 = \bar{w}xy\bar{z}$ ($i=6=0110$). Se construye con 4 inversores (para las versiones complementadas de cada entrada) y 16 compuertas AND de 4 entradas.

Verificación: por construcción, para cualquier entrada fija (w,x,y,z), solo el AND cuyo patrón de complementos coincide exactamente con esos valores da 1 — los otros 15 tienen al menos un literal en 0, así que su AND da 0. Exactamente una salida en 1 por cada combinación de entrada. ✓

5.2 MULTIPLEXOR (MUX)

Definición del curso — cuidado con esto: un multiplexor selecciona una de 2^n entradas de **datos** ($D_0 \dots D_{2^n-1}$) hacia una única salida, usando **n líneas de control (selección)**. Cuando el enunciado habla de “un multiplexor de k entradas”, **k se refiere a las entradas de control**, no a las de datos — un MUX de “2 entradas de control” tiene 4 líneas de datos (4:1), uno de “3 entradas de control” tiene 8 líneas de datos (8:1).

Error de tolerancia cero: confundir “cantidad de entradas” (control) con “cantidad de líneas de datos” invierte todo el diseño — si el enunciado pide un MUX de n entradas y armás el circuito con 2^n líneas de control en vez de n , el circuito completo queda mal, no es un detalle menor.

5.3 DEMULTIPLEXOR

Definición: la operación inversa del MUX — una única entrada de datos se dirige hacia una de 2^n salidas, según el valor de n líneas de control. La salida $Y_i = \text{entrada} \cdot (\text{AND de control que selecciona } i)$ — estructuralmente es un decodificador donde cada AND recibe además la entrada de datos como una entrada más.

5.4 MULTIPLEXOR COMO GENERADOR DE FUNCIONES (MUX COMO LOOK-UP TABLE)

Idea central — muy recurrente en examen: cualquier función de n variables se puede implementar con un único multiplexor de $2^n:1$, sin usar ninguna otra compuerta:

1. Conectar $n-1$ variables a las líneas de **selección** del MUX ($2^{n-1}:1$) — o las n variables completas a un MUX de $2^n:1$ si no se busca reducir el tamaño.
2. Para cada entrada de datos D_i , mirar en la tabla de verdad las filas donde las variables de selección toman el patrón i , y cablear D_i al valor que le corresponde: 0, 1, la variable restante, o su complemento.

Este método se desarrolla completo con verificación en la sección 7.

6. Sumador completo de 1 bit y sumador de n bits

6.1 HALF ADDER (SEMISUMADOR)

Definición: suma dos bits a, b sin considerar acarreo de entrada.

$$S = a \oplus b \quad C = a \cdot b$$

Verificación (4 filas de a, b): $(0,0) \rightarrow S=0, C=0$ ($0+0=00$ ✓) · $(0,1) \rightarrow S=1, C=0$ ($0+1=01$ ✓) · $(1,0) \rightarrow S=1, C=0$ ($1+0=01$ ✓) · $(1,1) \rightarrow S=0, C=1$ ($1+1=10$ ✓). Coincide con la suma aritmética en las 4 filas.

6.2 FULL ADDER (SUMADOR COMPLETO)

Definición: suma dos bits a, b **más** un acarreo de entrada c_{in} , produciendo suma S y acarreo de salida c_{out} .

$$S = a \oplus b \oplus c_{in} \quad c_{out} = a \cdot b + c_{in} \cdot (a \oplus b) = a \cdot b + a \cdot c_{in} + b \cdot c_{in}$$

Construcción: dos half adders en cascada (el primero suma $a+b$, el segundo suma ese resultado con c_{in}) más una compuerta OR para combinar los dos acarreos parciales.

Verificación numérica (se comprobaron las 8 combinaciones de a, b, c_{in} contra la suma aritmética real $a+b+c_{in}$): por ejemplo $a=1, b=1, c_{in}=1 \rightarrow$ suma real $=3=11_2 \rightarrow S=1, c_{out}=1$. Con la fórmula: $S=1 \oplus 1 \oplus 1=1 \checkmark$; $c_{out}=(1 \cdot 1) + (1 \cdot (1 \oplus 1))=1+0=1 \checkmark$. Verificado exhaustivamente en las 8 filas: S siempre coincide con el bit menos significativo de $a+b+c_{in}$, y c_{out} con el acarreo real (1 si $a+b+c_{in} \geq 2$).

Truco de verificación cruzada para c_{out} : $c_{out}=1$ exactamente cuando ****al menos dos**** de las tres entradas (a, b, c_{in}) son 1 — es literalmente la función mayoría de 3 variables. Si tu expresión de c_{out} no cumple “1 si al menos 2 de 3 entradas son 1”, está mal.

6.3 SUMADOR DE N BITS EN CASCADA (RIPPLE CARRY)

Construcción: n full adders encadenados, donde el acarreo de salida c_{out} del bit i se conecta como acarreo de entrada c_{in} del bit $i+1$ (el bit 0, el menos significativo, recibe $c_{in}=0$). Las entradas son los bits a_i, b_i de cada número; las salidas son los bits de suma S_i más el acarreo final c_{out} del último full adder (que indica desborde sin signo).

Error frecuente: olvidar que el ripple carry tiene ****latencia de propagación**** — el acarreo tiene que “viajar” secuencialmente de bit 0 a bit $n-1$ antes de que el resultado sea válido, así que el tiempo de cómputo del sumador completo crece con n (no es instantáneo aunque cada full adder individual sí lo sea). Este es el motivo por el que existen sumadores más rápidos (carry look-ahead), fuera del alcance de esta guía pero mencionado seguido en examen como pregunta teórica.

7. Caso resuelto — MUX como look-up table (verificado completo)

Función elegida: $f(x,y,z) = \sigma(3,4,6,7)$, con x,y como líneas de selección de un MUX 4:1 y z como variable “libre” que se cablea a las entradas de datos.

Paso 1 — Mapa de Karnaugh (filas x , columnas yz):

$x \backslash yz$	00	01	11	10
0	0	0	1	0
1	1	0	1	1

Paso 2 — tabla de conexión al MUX (D_i indexado por xy , según lo que vale f en cada fila para $z=0$ y $z=1$):

xy (índice i)	$f(z=0)$	$f(z=1)$	D_i a cablear
00 (D_0)	0	0	0 (constante)
01 (D_1)	0	1	z
10 (D_2)	1	0	$\bar{z}$
11 (D_3)	1	1	1 (constante)

Verificación de que K-map y tabla de MUX coinciden: por ejemplo para $xy=01$ (fila $x=0$, columnas $yz=01$ y $yz=11$ del mapa): K-map da $f=0$ en $yz=01$ ($z=1$... hay que leer con cuidado el orden Gray de columnas: $yz=01 \rightarrow y=0, z=1$, y $yz=11 \rightarrow y=1, z=1$). Repasando por índice binario directo xyz en vez del mapa (más seguro para no confundir el orden Gray de columnas): minterms $\{3,4,6,7\}$ en binario xyz son 011, 100, 110, 111.

$xy=00$: minterms con $xy=00 \rightarrow$ ninguno (000, 001 no están) $\rightarrow D_0=0$ ✓

$xy=01$: minterms con $xy=01 \rightarrow$ 011 ($z=1$) sí, 010 no $\rightarrow D_1=z$ ✓ (vale 1 solo si $z=1$)

$xy=10$: minterms con $xy=10 \rightarrow$ 100 ($z=0$) sí, 101 no $\rightarrow D_2=\bar{z}$ ✓ (vale 1 solo si $z=0$)

$xy=11$: minterms con $xy=11 \rightarrow$ 110 y 111, ambos $\rightarrow D_3=1$ ✓ (vale 1 siempre)

Coincide exactamente con la tabla de conexión. ✓ Verificado además por tabla de verdad completa (8 filas de x,y,z): el circuito MUX así cableado reproduce $f=1$ exactamente en las filas 3,4,6,7 y $f=0$ en el resto.

Observación pedagógica clave: la implementación por MUX ****no necesita ninguna simplificación algebraica ni Karnaugh**** — el cableado sale directo de leer la tabla de verdad fila por fila agrupada de a 2 (por la variable no usada en selección). De hecho, la expresión algebraica mínima de esta misma función es $y \cdot z + x \cdot \bar{z}$ (2 términos, verificado independientemente) — más corta que el circuito MUX, que usa las 4 líneas de datos sin simplificar. Esto ilustra que MUX-como-LUT prioriza ****rapidez de diseño****, no mínimo número de compuertas: son dos objetivos distintos y el enunciado del ejercicio determina cuál se pide.

Error frecuente en este tipo de ejercicio: al armar la tabla de conexión, mezclar el orden de las variables de selección (¿x es el bit alto o bajo del índice i?) entre la tabla de verdad y el datasheet del MUX. Definir explícitamente “índice $i = x \cdot 2 + y$ ” (o el orden que corresponda) ****antes**** de armar la tabla, y mantenerlo fijo, evita el error más común de este ejercicio.

8. Ejercicios de repaso rápido (con respuesta)

Tapá la columna de respuesta y resolvé antes de mirar.

Enunciado	Respuesta
¿Qué axioma justifica $a \cdot (b+c) = a \cdot b + a \cdot c$?	ax.5, distributiva
¿Qué axioma justifica $a + \bar{a} = 1$?	ax.6, complemento
Demostrar $a + a \cdot b = a$ en una línea usando el resultado de $a \cdot (a+b) = a$	$a + ab = a \cdot (a+ab)$ por idempotencia previa... (ver demostración completa en sección 1; no se puede saltar el paso citando solo “absorción” si el enunciado pide axiomas)
$f(a,b,c) = \sigma(0,3,5)$ — escribir suma de productos canónicos	$\bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c} + a\bar{b}c$
$g(a,b,c) = \pi(1,2)$ — escribir producto de sumas canónicas	$(a+\bar{b}+c) \cdot (a+b+\bar{c})$
¿Karnaugh garantiza mínima cantidad de compuertas físicas?	Falso — garantiza mínimos términos/literales en 2 niveles, no compuertas físicas
¿Puede haber dos expresiones mínimas distintas para la misma tabla de verdad?	Sí — el mínimo en 2 niveles no es único (Ejercicio 6, sección 3.3)
Construir NOT usando solo NAND	$\text{NAND}(A,A) = (A \cdot A) = \bar{A}$
MUX de “n entradas” en la definición del curso — ¿n es de control o de datos?	de control (líneas de selección)
MUX 8:1 usado como look-up table — ¿cuántas variables de la función caben usando las 3 líneas de selección + 1 variable libre en los datos?	4 variables
Full adder: ¿cuándo vale 1 el acarreo de salida?	cuando al menos 2 de las 3 entradas (a, b, c_{in}) valen 1
Ripple carry de n bits: ¿por qué es más lento que un full adder solo?	el acarreo se propaga secuencialmente bit a bit, agregando latencia proporcional a n
Decodificador $3 \rightarrow 8$: expresión de Y_5 con entradas a,b,c	$\bar{a}\bar{b}c$ ($5=101_2$)
¿{NOR} es un conjunto lógicamente completo?	Sí — con NOR se puede construir NOT, y luego AND y OR (dual de NAND)

9. Resumen ultra-rápido (2 minutos)

- **Axiomas (citar por número):** ax.1 cierre, ax.2 neutro, ax.3 conmutativa, ax.4 asociativa, ax.5 distributiva, ax.6 complemento. Teoremas derivados: idempotencia, dominación, absorción, involución, De Morgan, consenso.
- Σ : suma de minterms donde $f=1$ (variable sin complementar si vale 1). Π : producto de maxterms donde $f=0$ (variable sin complementar si vale 0). Los índices de Σ y Π de una misma función son complementarios entre 0 y 2^n-1 .
- **Karnaugh:** agrupar 1's en bloques potencia de 2, con wrap-around, leyendo solo las variables constantes del grupo. El mínimo en 2 niveles **no es único** (mismo costo, distinto agrupamiento posible).
- **Karnaugh minimiza términos/literales, NO garantiza mínima cantidad de compuertas físicas** — pregunta clásica de teórico.
- **{NAND} y {NOR} son conjuntos lógicamente completos:** con solo NAND se construyen NOT, AND y OR (demostración vía De Morgan + involución + idempotencia).
- **Decodificador:** n entradas, 2^n salidas, cada $Y_i = \text{AND}$ de las entradas complementadas según el bit i .
- **Multiplexor:** “ n entradas” en la definición del curso = líneas de **control**, no de datos (2^n líneas de datos). MUX como look-up table: implementa cualquier función de n variables cableando 0, 1, la variable libre o su complemento en cada D_i , sin necesidad de simplificar.
- **Demultiplexor:** operación inversa del MUX — una entrada de datos dirigida a una de 2^n salidas según control.
- **Half adder:** $S=a \oplus b$, $C=a \cdot b$. **Full adder:** $S=a \oplus b \oplus c_{in}$, $c_{out}=ab+ac_{in}+bc_{in}$ (vale 1 si ≥ 2 de las 3 entradas son 1). **Ripple carry:** n full adders en cascada, latencia proporcional a n .